

App User Behavior Segmentation Project

Complete Code Documentation & Interview Guide

Python | KMeans Clustering | Streamlit | Data Science

Table of Contents

1. Project Overview
2. 01_data_loading.py — Load & Inspect Data
3. 02_data_cleaning.py — Clean & Standardize
4. 03_EDA.py — Exploratory Data Analysis
5. 04_feature_scaling.py — Standardize Features (StandardScaler)
6. 05_kmeans_clustering.py — KMeans Clustering & PCA Visualization
7. 06_business_insights.py — Business Insights & Recommendations
8. 07_streamlit_app.py — Interactive Streamlit Dashboard
9. Interview Questions — 30 Detailed Q&A;

1. Project Overview

This project is a complete end-to-end **Data Science and Machine Learning pipeline** that analyses mobile app user behaviour and groups users into meaningful segments using the **KMeans unsupervised clustering algorithm**. The dataset contains behavioural and demographic attributes for thousands of app users, including how frequently they open the app, how long they stay active, how many notifications they open, their engagement and churn risk scores, subscription plan type, and the device they use.

The pipeline is structured into seven modular Python scripts, each handling a distinct stage of the workflow. It begins with **01_data_loading.py**, which loads the raw CSV and prints a structural overview of the dataset including shape, column types, summary statistics, and missing value counts — giving the developer an immediate understanding of what they are working with. Next, **02_data_cleaning.py** addresses data quality issues: the only column with missing values (`rating_given`) is imputed using the median rather than the mean, making the fill value robust against outliers, and all categorical text columns are stripped of whitespace and converted to Title Case to ensure consistent grouping.

The third script, **03_EDA.py**, performs Exploratory Data Analysis by generating ten saved visualisations — histograms for engagement, churn risk, sessions, and login recency; bar charts for subscription and device distribution; scatter plots revealing the relationship between sessions and active time as well as engagement versus churn risk; and a full Pearson **correlation heatmap** that highlights which features are strongly related to each other. These charts form the analytical backbone of the project report.

Before machine learning can begin, **04_feature_scaling.py** selects fourteen behavioural features and applies **StandardScaler** from scikit-learn. This step is critical because KMeans relies on Euclidean distance — if one feature has values in the hundreds and another in single digits, the larger feature will dominate cluster assignments. Scaling transforms every feature to have a mean of zero and a standard deviation of one, ensuring all features contribute equally to the distance calculation.

The core modelling happens in **05_kmeans_clustering.py**. First, the **Elbow Method** iterates KMeans from $K=1$ to $K=10$, plotting inertia (total within-cluster variance) at each step. The sharp bend in the curve at $K=4$ confirms four clusters as the optimal choice. KMeans is then fitted with $K=4$, assigning each user to one of four groups. Clusters are profiled by computing their average engagement score, and sorted from highest to lowest to assign human-readable names: *High Engagement*, *Moderate Engagement*, *Low Engagement*, and *Occasional Users*. **PCA (Principal Component Analysis)** then compresses the 14-dimensional scaled data into two principal components so the four clusters can be plotted in a 2D scatter chart, visually confirming that the groups are genuinely distinct.

The insights script **06_business_insights.py** translates cluster statistics into actionable business language. Using groupby aggregations and cross-tabulations, it reports the total user count, average engagement, average churn risk, average sessions, and average days since last login per cluster — then maps specific retention and growth strategies to each segment. Finally, **07_streamlit_app.py** packages everything into an interactive web dashboard. Users can filter by segment, device type, and subscription plan via sidebar widgets. The page displays live-updating **KPI metric cards** and nine matplotlib charts rendered inline, plus a business recommendations section using Streamlit's colour-coded alert boxes. The `@st.cache_data` decorator ensures the CSV is read only once no matter how many times the user adjusts filters.

2. 01_data_loading.py — Load & Inspect Data

This is the very first script executed in the project pipeline. Its sole responsibility is to load the raw dataset from a CSV file into a Pandas DataFrame and print four diagnostic summaries: the shape (rows × columns), the first five rows for a data preview, the column data types and non-null counts via `df.info()`, the descriptive statistics (mean, std, min, max, quartiles) via `df.describe()`, and the per-column missing value count via `df.isnull().sum()`. Running this script before any processing is a best practice — it allows the developer to understand the data structure, spot data-type mismatches, identify missing values, and detect potential outliers before committing to a cleaning strategy.

```
import pandas as pd
# pandas is the primary data manipulation library in Python data science
# 'pd' is the universally accepted alias – always import it as pd by convention
import numpy as np
# numpy provides fast numerical array operations; imported here as a standard companion

df = pd.read_csv('app_user_behavior_dataset.csv')
# pd.read_csv() parses a comma-separated file and returns a DataFrame object
# A DataFrame is a 2D labelled table – rows are records, columns are features
# 'df' is the standard variable name convention for a DataFrame

print('Shape:', df.shape)
# df.shape is a tuple attribute: (number_of_rows, number_of_columns)
# e.g. (50000, 22) means 50,000 users and 22 feature columns

print(df.head())
# df.head() returns the first 5 rows by default – pass a number for more: df.head(10)
# Useful for a quick sanity check that data loaded correctly with right column names

print(df.info())
# df.info() prints: column names, count of non-null values, and data type (dtype)
# Helps detect columns wrongly loaded as 'object' instead of int/float

print(df.describe())
# df.describe() computes count, mean, std, min, 25%, 50%, 75%, max for numeric cols
# Great for spotting outliers – e.g. if max sessions_per_week is 9999, something is wrong

print(df.isnull().sum())
# df.isnull() returns a boolean DataFrame: True where value is NaN, False otherwise
# .sum() counts True values per column – gives exact count of missing values per feature
# Zero for all columns = no missing data; non-zero = cleaning required
```

3. 02_data_cleaning.py — Clean & Standardize

Data cleaning is the process of detecting and correcting errors and inconsistencies in raw data to improve its quality before analysis. In this project three cleaning operations are performed. First, the only column with missing values — `rating_given` — is imputed using the column median. The median is preferred over the mean here because it is not influenced by extreme outliers (a few users who gave very high or very low ratings would skew the mean significantly). Second, all duplicate rows are removed using `drop_duplicates()` to ensure that no user is counted twice, which could otherwise inflate engagement metrics for certain clusters. Third, five categorical text columns are standardised: whitespace is trimmed and values are converted to Title Case so that entries like 'android', ' Android ', and 'ANDROID' are treated as the same category, preventing split groups in later analysis. The cleaned DataFrame is then saved to a new CSV file so subsequent scripts always work from

the clean version.

```
import pandas as pd
import numpy as np

df = pd.read_csv('app_user_behavior_dataset.csv')
# Load the raw dataset – cleaning is always done on the original, never overwrite raw data

# ---- STEP 1: Handle Missing Values ----
df['rating_given'] = df['rating_given'].fillna(df['rating_given'].median())
# df['rating_given'].median() calculates the middle value of the non-null ratings
# .fillna() replaces every NaN in that column with the computed median value
# Using median (not mean) because ratings are discrete and median is outlier-resistant

# ---- STEP 2: Remove Duplicates ----
print('Duplicates before:', df.duplicated().sum())
# df.duplicated() marks True for every row that is an exact copy of a previous row
# .sum() counts all True values – tells us how many duplicate rows exist
df = df.drop_duplicates()
# drop_duplicates() removes duplicate rows, keeping the first occurrence by default
# Reassigning back to df overwrites the old DataFrame with the deduplicated one
print('Duplicates after:', df.duplicated().sum())
# Should print 0 – confirming all duplicates have been removed

# ---- STEP 3: Standardize Categorical Columns ----
cat_cols = ['gender', 'country', 'device_type', 'subscription_type', 'marketing_source']
# List of columns whose values are text categories that may have inconsistent formatting

for col in cat_cols:
    df[col] = df[col].str.strip().str.title()
# Iterate over each categorical column and apply two chained string operations:
# str.strip() – removes spaces from both ends of the string value
# str.title() – capitalises the first letter of each word
# Result: ' free plan ' -> 'Free Plan', 'ANDROID' -> 'Android'

print(df.isnull().sum())
# Verify no missing values remain after cleaning

df.to_csv('cleaned_data.csv', index=False)
# Save the clean DataFrame to a new file; index=False omits the row number column
# This file is the input for all subsequent scripts (EDA, scaling, clustering)
```

4. 03_EDA.py — Exploratory Data Analysis

EDA (Exploratory Data Analysis) is a crucial phase where the analyst visually and statistically summarises the cleaned dataset to uncover patterns, distributions, outliers, and relationships before building any model. Skipping EDA is a common mistake — it often leads to poor model choices or overlooked data issues. This script creates ten distinct charts covering every important feature in the dataset.

The first three charts are histogram distributions for `engagement_score`, `churn_risk_score`, and `sessions_per_week`, showing how users are spread across those behavioural metrics. Charts four and five are bar and pie charts for subscription and device type, revealing the demographic composition of the user base. Chart six is a scatter plot of engagement vs churn risk — a key business chart that shows whether high-engagement users also have low churn (the expected inverse relationship). Chart seven plots sessions vs daily active minutes to confirm that more frequent users also spend more time per day. Chart eight is a grouped bar chart showing average engagement by subscription type — revealing whether premium subscribers are more engaged. Chart nine displays the distribution of days since last login, identifying how dormant the user base is. The tenth and most comprehensive chart is a Pearson correlation heatmap that shows the pairwise correlation between all numeric features, helping identify redundant or highly correlated features before clustering.

```
import matplotlib.pyplot as plt
# matplotlib.pyplot is the core Python plotting library; plt is the standard alias
import seaborn as sns
# seaborn wraps matplotlib with a higher-level API for statistical charts
# It handles color palettes, axis labels, and styling more elegantly than raw matplotlib
sns.set_style('whitegrid')
# Applies a global theme: white background with faint grey grid lines for all charts
# Other options: 'darkgrid', 'ticks', 'white', 'dark'
plt.rcParams['figure.figsize'] = (10, 6)
# rcParams is matplotlib's global settings dictionary
# Setting figure.figsize makes every subsequent figure 10 inches wide by 6 inches tall
# ---- Chart 1: Engagement Distribution ----
plt.figure()
# Creates a new blank figure; without this, matplotlib draws on the previous chart
sns.histplot(df['engagement_score'], bins=30, color='steelblue')
# histplot divides the range of values into 30 equal-width bins and counts users per bin
# bins=30 gives enough granularity to see the distribution shape without over-smoothing
plt.tight_layout()
# Automatically adjusts spacing between plot elements to prevent overlapping labels
plt.savefig('output/01_engagement_distribution.png')
# Writes the figure to a PNG image file at the specified path
plt.close()
# Closes the figure and releases memory – critical when generating many charts in a loop
# ---- Chart 10: Correlation Heatmap ----
numeric_cols = df.select_dtypes(include=[np.number]).columns
# select_dtypes filters columns by data type; np.number includes both int and float
# Excludes text columns (gender, device_type etc.) which cannot be correlated numerically
corr_matrix = df[numeric_cols].corr()
# .corr() computes pairwise Pearson correlation for all numeric column pairs
```

```
# Returns a square matrix: corr[A][B] = how linearly related A and B are
# +1 = perfect positive, -1 = perfect negative, 0 = no linear relationship
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5)
# annot=True overlays correlation values on each cell
# fmt='.2f' formats them to 2 decimal places
# cmap='coolwarm': blue tiles = negative correlation, red tiles = positive correlation
# linewidths=0.5 adds thin separating lines between cells for readability
```

5. 04_feature_scaling.py — Standardize Features

Feature scaling is a preprocessing step that transforms numerical features to a common scale before feeding them into distance-based algorithms like KMeans. Without scaling, features with large numerical ranges dominate the Euclidean distance calculation. For example, `daily_active_minutes` can range from 0 to 500 while `social_shares` might range from 0 to 10 — an unscaled KMeans would treat a one-unit difference in `daily_active_minutes` as roughly equivalent to 50 units in `social_shares`, which is incorrect.

`StandardScaler` (also called Z-score normalisation) is applied here. For each feature it computes the mean and standard deviation from the training data, then transforms every value using the formula: $z = (x - \text{mean}) / \text{std}$. After scaling, every feature has a mean of exactly 0 and a standard deviation of 1, so they all contribute equally to distance calculations. Fourteen behavioural features are selected for clustering — columns like `user_id`, `age`, `gender`, and `country` are deliberately excluded because they are either identifiers or demographic attributes that do not represent how a user actually behaves inside the app.

```
from sklearn.preprocessing import StandardScaler
# sklearn (scikit-learn) is Python's main ML library; preprocessing contains scaling tools
# StandardScaler implements Z-score normalisation:  $z = (x - \text{mean}) / \text{std}$ 

features = [
    'sessions_per_week', 'avg_session_duration_min', 'daily_active_minutes',
    'feature_clicks_per_session', 'notifications_opened_per_week',
    'in_app_search_count', 'pages_viewed_per_session', 'days_since_last_login',
    'content_downloads', 'social_shares', 'churn_risk_score',
    'engagement_score', 'rating_given', 'ads_clicked_last_30_days'
]
# 14 behavioural features selected – user_id/age/gender excluded as non-behavioural
# These capture: frequency, depth, recency, engagement, and monetisation behaviour
X = df[features]
# X is the feature matrix – shape: (n_users, 14)
# Selecting only the relevant columns creates a clean input for the scaler
scaler = StandardScaler()
# Instantiate the scaler object – no computation yet, just creating the transformer
X_scaled = scaler.fit_transform(X)
# fit(): computes mean and std for each of the 14 columns from X
# transform(): applies  $z = (x - \text{mean}) / \text{std}$  to every value in the matrix
# fit_transform() does both in one call and returns a numpy array
print('Before scaling - engagement_score mean:', round(df['engagement_score'].mean(), 2))
print('After scaling - mean should be ~0:', round(X_scaled[:,
    features.index('engagement_score')].mean(), 4))
# Verifying scaling worked: post-scaling mean should be ~0.0 for every feature
# features.index('engagement_score') gets the column position (integer index) in the array
```

```
X_scaled_df = pd.DataFrame(X_scaled, columns=features)
# Convert the numpy array back to a DataFrame so column names are retained
# Without this step the CSV would have unnamed columns (0, 1, 2 ...)
X_scaled_df.to_csv('scaled_data.csv', index=False)
# Persist scaled data so the clustering script can read it without re-scaling
```

6. 05_kmeans_clustering.py — KMeans & PCA Visualization

This is the most important script in the project — it contains the core machine learning logic. KMeans is an iterative unsupervised algorithm that partitions users into K non-overlapping clusters by minimising the total within-cluster variance (inertia). The algorithm begins by randomly placing K centroids in the feature space, assigns every user to the nearest centroid using Euclidean distance, recalculates each centroid as the mean of all points in its cluster, and repeats until assignments no longer change. The parameter `n_init=10` runs this entire process ten times with different random starting centroids and keeps the result with the lowest inertia, avoiding locally optimal but globally poor solutions.

The **Elbow Method** is used to determine the optimal K. Running KMeans for K=1 to K=10 and plotting the resulting inertia values produces a curve that drops steeply at first and then flattens out. The 'elbow' — the point of maximum curvature — indicates that adding more clusters beyond that K provides diminishing improvement. In this project the elbow is observed at K=4, which also aligns with the business requirement of four distinct user personas.

After clustering, each user receives a cluster label (0–3). To assign meaningful names, the four clusters are sorted by their average engagement_score from highest to lowest, and named High Engagement, Moderate Engagement, Low Engagement, and Occasional Users in that order. **PCA (Principal Component Analysis)** is then applied to produce a 2D scatter plot — since 14 features cannot be visualised simultaneously, PCA finds the two linear combinations of features that capture the most variance and projects all users onto that 2D plane. If the four cluster blobs in the PCA plot are visually well-separated, it validates that the clustering is meaningful.

```
from sklearn.cluster import KMeans
# KMeans: unsupervised partitioning algorithm from scikit-learn's cluster module
from sklearn.decomposition import PCA
# PCA: linear dimensionality reduction technique from scikit-learn's decomposition module
# ---- ELBOW METHOD ----
inertia = []
# Empty list to collect inertia (total within-cluster sum of squared distances) for each K
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    # n_clusters=k: number of clusters to form
    # random_state=42: sets the random seed so results are reproducible every run
    # n_init=10: runs the algorithm 10 times with different centroid seeds; picks best
    kmeans.fit(X_scaled)
    # .fit() trains KMeans: assign points -> update centroids -> repeat until convergence
    inertia.append(kmeans.inertia_)
    # .inertia_ attribute: sum of squared distances of each point to its cluster centroid
# ---- FINAL K=4 MODEL ----
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X_scaled)
# fit_predict(): fits the model AND returns cluster label array in one step
# Each user gets an integer label: 0, 1, 2, or 3
# ---- CLUSTER PROFILING ----
cluster_profile = df.groupby('cluster')[['sessions_per_week', 'engagement_score',
    'churn_risk_score', 'days_since_last_login']].mean().round(2)
# groupby groups rows sharing the same cluster ID, then .mean() averages each column
```

```

# .round(2) limits output to 2 decimal places for clean printing
# ---- NAME CLUSTERS ----
cluster_engagement = df.groupby('cluster')['engagement_score'].mean()
sorted_clusters = cluster_engagement.sort_values(ascending=False).index.tolist()
# sort_values(ascending=False) orders clusters from highest to lowest engagement
# .index.tolist() extracts just the cluster number sequence e.g. [2, 0, 3, 1]
labels = ['High Engagement', 'Moderate Engagement', 'Low Engagement', 'Occasional Users']
cluster_names = {}
for i, cluster in enumerate(sorted_clusters):
    cluster_names[cluster] = labels[i]
# enumerate() gives both index (i) and value (cluster) in each loop iteration
# Builds a dict like {2: 'High Engagement', 0: 'Moderate Engagement', ...}
df['cluster_name'] = df['cluster'].map(cluster_names)
# .map(dict) replaces each value in the column with the corresponding dict value
# ---- PCA VISUALIZATION ----
pca = PCA(n_components=2)
# n_components=2: project 14-dimensional data down to 2 dimensions
X_pca = pca.fit_transform(X_scaled)
# fit(): finds the 2 principal components that explain maximum variance
# transform(): projects every user onto those 2 components
df['pca1'] = X_pca[:, 0]
df['pca2'] = X_pca[:, 1]
#[:, 0] selects all rows of column index 0 (first principal component)
# These are stored in df for use in the scatter plot
for i, name in enumerate(['High Engagement', 'Moderate Engagement',
    'Low Engagement', 'Occasional Users']):
    mask = df['cluster_name'] == name
    # Boolean mask: True for rows belonging to this cluster, False for others
    plt.scatter(df[mask]['pca1'], df[mask]['pca2'],
        label=name, alpha=0.3, color=colors[i], s=10)
# alpha=0.3: 70% transparent so overlapping points are still visible
# s=10: small marker size to handle thousands of overlapping data points

```

7. 06_business_insights.py — Insights Report

The business insights script bridges the gap between technical cluster statistics and real-world decision-making. After clustering, the raw group labels (0, 1, 2, 3) are not useful to stakeholders — what a product manager or marketing team needs are clear profiles: how many users are in each group, how engaged they are, what their churn risk looks like, and which subscription plan and device type they favour. This script provides exactly that through structured aggregations and cross-tabulations.

The `.agg()` method with named aggregation syntax is used to compute five metrics per cluster in a single operation, which is far more efficient than writing separate `groupby` calls for each metric. The `pd.crosstab()` function creates frequency tables showing how subscription types and device types are distributed across clusters — for example, it might reveal that Low Engagement users are disproportionately on the Free plan and use Android, which would inform a targeted mobile-first re-engagement campaign. The script ends with a structured recommendations block mapping each cluster to a specific business strategy: loyalty programs for High Engagement users, personalised push notifications for Moderate users, win-back emails for Low

Engagement users, and feature reminders for Occasional users.

```
df = pd.read_csv('clustered_data.csv')
# Load the final dataset that already contains cluster and cluster_name columns
summary = df.groupby('cluster_name').agg(
    Total_Users=('user_id', 'count'),
    Avg_Engagement=('engagement_score', 'mean'),
    Avg_Churn_Risk=('churn_risk_score', 'mean'),
    Avg_Sessions=('sessions_per_week', 'mean'),
    Avg_Daily_Minutes=('daily_active_minutes', 'mean'),
    Avg_Days_Since_Login=('days_since_last_login', 'mean')
).round(2)
# Named aggregation syntax: output_col_name=('source_col', 'aggregation_function')
# All 6 metrics computed in one groupby call – efficient and readable
# .round(2) applied to entire DataFrame result at once
high = df[df['cluster_name'] == 'High Engagement']
# Boolean indexing: filter rows where the cluster_name equals the target string
print(f'HIGH ENGAGEMENT USERS: {len(high)}')
print(f'Avg Engagement Score: {high["engagement_score"].mean():.2f}')
# f-string with :.2f rounds the float to exactly 2 decimal places in the output
# Escaped quotes \" needed inside f-string because outer quotes are double quotes
sub_cluster = pd.crosstab(df['cluster_name'], df['subscription_type'])
# pd.crosstab creates a frequency cross-tabulation
# Rows = cluster names, Columns = subscription types, Cells = user count
# Reveals which subscription types are over/under-represented in each cluster
device_cluster = pd.crosstab(df['cluster_name'], df['device_type'])
# Same pattern for device type – helps target device-specific UI/UX improvements
# e.g. if Occasional Users are mostly tablet users, redesign tablet onboarding flow
```

8. 07_streamlit_app.py — Interactive Streamlit Dashboard

Streamlit is an open-source Python framework that converts a regular Python script into a fully interactive web application with zero HTML, CSS, or JavaScript. It works on a reactive execution model: the entire script re-runs from top to bottom every time the user interacts with any widget — a sidebar filter, a dropdown, or a slider. This makes building data dashboards extremely fast since the developer just writes Python and Streamlit handles all the frontend rendering.

The dashboard is structured in four main sections. The **KPI cards** at the top show five live-updating metrics — total users, average engagement, average churn risk, average sessions per week, and average daily active minutes — all computed from the filtered DataFrame. The **User Segmentation Analysis** section displays three charts side by side: a pie chart of cluster distribution, a bar chart of average engagement by cluster, and a bar chart of average churn risk by cluster. The **User Behaviour Analysis** section shows device type distribution, subscription type distribution, and a scatter plot of sessions vs engagement. Finally, the **Business Insights** section shows a cluster summary table alongside colour-coded recommendation cards.

The `@st.cache_data` decorator is applied to the `load_data()` function to prevent the CSV from being re-read on every user interaction. Without caching, a dashboard with many filters would re-read the file hundreds of times per session, causing significant lag. The sidebar multiselect widgets use `.unique()` to dynamically populate options from the actual data, so the dashboard adapts automatically if new cluster types or device categories appear in the dataset. The `filtered_df` variable is a live view — all downstream charts and metrics automatically reflect the current filter state.

```
import streamlit as st
# streamlit is the web app framework; st is the universal alias
st.set_page_config(page_title='App User Behavior Segmentation',
page_icon='■', layout='wide')
# MUST be the very first Streamlit command in the script
# layout='wide' uses the full browser width instead of a narrow centred column
@st.cache_data
# @st.cache_data caches the return value of the function after the first call
# On subsequent script re-runs (triggered by widget interactions), cached result is
returned
# Without this, load_data() would re-read the CSV file on every single user click
def load_data():
    return pd.read_csv('clustered_data.csv')
df = load_data()
cluster_filter = st.sidebar.multiselect(
    'Select User Segment',
    options=df['cluster_name'].unique(),
    default=df['cluster_name'].unique()
)
# st.sidebar places the widget in the left-side panel instead of the main page
# multiselect renders a dropdown where users can select multiple options simultaneously
# options= populates the dropdown; default= pre-selects all options on first load
# df['cluster_name'].unique() returns an array of distinct values from the column
filtered_df = df[
(df['cluster_name'].isin(cluster_filter)) &
```

```

(df['device_type'].isin(device_filter)) &
(df['subscription_type'].isin(subscription_filter))
]
# .isin(list) returns True for each row whose value appears in the given list
# & operator combines boolean conditions with AND logic (all three must be True)
# This creates a filtered view that updates live with every sidebar change
col1, col2, col3, col4, col5 = st.columns(5)
# st.columns(n) divides the page into n equal-width side-by-side containers
# Assigning to variables lets us place content in specific columns
col1.metric('Total Users', f'{len(filtered_df):,}')
# st.metric() renders a styled card with a label (top) and value (large text below)
# f'{len(filtered_df):,}' – the , format spec adds thousands separator: 49832 -> '49,832'

with col1:
    fig, ax = plt.subplots()
    # with col1: context manager places everything inside into column 1
    # plt.subplots() returns a figure object (fig) and an axes object (ax)
    # fig is the whole canvas; ax is the drawing area where charts are placed
    cluster_counts.plot(kind='pie', autopct='%1.1f%%', ax=ax)
    # autopct='%1.1f%%' adds percentage labels like '27.3%' directly onto pie slices
    st.pyplot(fig)
    # st.pyplot(fig) renders the matplotlib figure inline in the Streamlit page
    plt.close()
    # Always close after rendering to release memory – especially with multiple charts
    st.success('High Engagement Users: Offer loyalty rewards and premium subscriptions!')
    # st.success() renders a green alert box – used to signal positive/growth action
    st.info('Moderate Engagement: Send personalized push notifications!')
    # st.info() = blue box; st.warning() = orange; st.error() = red alert box
    # These colour-coded boxes communicate priority/urgency to business stakeholders

```

9. Interview Questions — 30 Detailed Q&A;

Section A: Python Fundamentals

Q1. What is a decorator in Python? Explain `@st.cache_data` with an example.

A decorator is a higher-order function that wraps another function to add behaviour before or after it executes, without modifying its source code. The syntax `@decorator_name` above a function definition is syntactic sugar for `func = decorator(func)`. In this project, `@st.cache_data` wraps `load_data()` so that the first time it is called, Streamlit executes the function body and stores the returned DataFrame in a cache. Every subsequent call — triggered by user widget interactions — retrieves the cached result instantly instead of re-reading the CSV from disk, making the dashboard dramatically faster.

Q2. Explain list comprehensions vs for-loops in Python. Where could they be used here?

A list comprehension is a concise, single-line way to create a list: `[expression for item in iterable if condition]`. It is generally faster than an equivalent for-loop because it is optimised at the C level inside CPython. In this project the for-loop that builds `cluster_names` could be written as a dict comprehension: `cluster_names = {cluster: labels[i] for i, cluster in enumerate(sorted_clusters)}`. List/dict comprehensions are preferred for simple transformations; explicit for-loops are better when the body is complex or has side effects.

Q3. What is the difference between `.loc[]` and `.iloc[]` in Pandas?

`.loc[]` (label-based) selects rows and columns by their label names: `df.loc[0, 'engagement_score']` selects the row with index label 0 and the column named 'engagement_score'. `.iloc[]` (integer position-based) selects by integer position regardless of labels: `df.iloc[0, 5]` selects the first row and sixth column. In this project, `X_scaled[:, features.index('engagement_score')]` uses NumPy's integer indexing equivalent — `features.index()` converts the column name to its integer position so the scaled array can be sliced correctly.

Q4. What does `enumerate()` do and why is it used instead of `range(len())`?

`enumerate(iterable)` returns an iterator of (index, value) tuples, providing both the position and the item simultaneously. `enumerate(sorted_clusters)` gives `(0, cluster_id_1)`, `(1, cluster_id_2)`, etc. Using `range(len(sorted_clusters))` and then indexing separately — `sorted_clusters[i]` — is more verbose and error-prone. `enumerate()` is the Pythonic approach and is slightly more memory-efficient since it does not materialise a separate index list.

Q5. What is the difference between `==` and `is` in Python?

`==` checks value equality — whether two objects have the same value. `is` checks identity — whether two variables point to the exact same object in memory. In this project, `df['cluster_name'] == 'High Engagement'` uses `==` to compare the string value of each cell with the string literal, producing a boolean Series. Using `is` here would not work correctly for string comparison against non-interned strings. A common bug is writing `if x is None` vs `if x == None` — the former is actually the correct pattern since `None` is a singleton.

Section B: Pandas & Data Processing

Q6. Explain the difference between `df.mean()`, `df.groupby().mean()`, and `.agg()`.

`df.mean()` computes the mean of every numeric column across all rows — a global average. `df.groupby('cluster_name').mean()` splits the DataFrame into groups by `cluster_name` and computes the mean of every numeric column within each group separately — giving a per-cluster profile. `.agg()` is more flexible: it allows applying different aggregation functions to different columns in one step, and supports custom function names via named aggregation syntax like `Total_Users=('user_id','count')`. In large datasets

.agg() is also more efficient than chaining multiple groupby operations.

Q7. What is pd.crosstab() and how does it differ from groupby?

pd.crosstab(index, columns) creates a frequency cross-tabulation table that counts the number of observations for each combination of two categorical variables. It is specifically designed for categorical summarisation. groupby().agg() is more general — it can apply any aggregation (mean, sum, custom functions) to any column type. In this project, pd.crosstab(df['cluster_name'], df['subscription_type']) quickly reveals how many Free, Premium, and Enterprise users are in each cluster, which would require a more verbose groupby with value_counts to replicate.

Q8. Why is median preferred over mean for imputing missing values in rating_given?

The mean is the arithmetic average of all values and is heavily influenced by outliers — if a few users gave extreme ratings of 1 or 10, the mean shifts away from the typical value. The median is the middle value when the data is sorted, which is entirely unaffected by extreme values. For a rating scale (typically 1–5), user ratings often have a skewed distribution with some outliers. Imputing with median preserves the central tendency without letting extreme values distort the filled-in data, which would otherwise bias downstream clustering.

Q9. What is the difference between drop_duplicates() and drop_na()?

drop_duplicates() removes rows that are exact copies of another row — detecting users who appear more than once in the dataset, which would inflate their contribution to cluster statistics. dropna() removes rows that contain at least one missing (NaN) value in any column, which could remove too much valid data if only one unimportant column is missing. In this project we used fillna(median) instead of dropna() because dropping rows with missing ratings would unnecessarily reduce the dataset size when the missing values can be reasonably estimated.

Q10. Explain boolean indexing in Pandas with an example from this project.

Boolean indexing filters a DataFrame by passing a boolean Series (True/False values) as the row selector. When you write df[df['cluster_name'] == 'High Engagement'], the inner expression df['cluster_name'] == 'High Engagement' returns a Series of True/False values — True for rows where the cluster name matches. Passing this Series back to df[] returns only the rows where the value is True. Combining multiple conditions requires wrapping each in parentheses and using & (AND) or | (OR) operators, as seen in the filtered_df construction with three simultaneous sidebar filter conditions.

Section C: Machine Learning & Statistics

Q11. Explain the KMeans algorithm step by step.

Step 1 — Initialisation: Randomly place K centroids in the feature space (or use KMeans++ smart initialisation, which is scikit-learn's default). Step 2 — Assignment: For each data point, compute the Euclidean distance to every centroid and assign the point to the nearest one. Step 3 — Update: Recalculate each centroid as the mean of all points currently assigned to it — the centroid moves to the geometric centre of its cluster. Step 4 — Convergence check: If any centroid moved in Step 3, go back to Step 2. If no centroid moved (or movement is below a tolerance), stop. The final cluster assignments are the output. In this project K=4 with n_init=10 (run 10 times, keep best).

Q12. What is inertia in KMeans and what does it measure?

Inertia (also called within-cluster sum of squares, WCSS) is the sum of squared Euclidean distances between each data point and its assigned cluster centroid. It measures how tightly packed (compact) the clusters are

— lower inertia means users within each cluster are more similar to each other. Inertia always decreases as K increases (more clusters = smaller, tighter groups), which is why it cannot be used alone to choose K — hence the Elbow Method plots inertia vs K and looks for the point where the rate of decrease sharply slows, indicating that additional clusters are no longer adding meaningful structure.

Q13. What is StandardScaler and what is the mathematical formula behind it?

StandardScaler implements Z-score normalisation. For each feature column, it computes the mean (μ) and standard deviation (σ) from the training data. Then every value x is transformed using: $z = (x - \mu) / \sigma$. The result z is called the Z-score — it represents how many standard deviations the original value is away from the mean. After transformation, every feature has mean = 0 and standard deviation = 1. This is different from MinMaxScaler which scales values to a fixed range like [0, 1] — StandardScaler is preferred for KMeans because it handles outliers more gracefully.

Q14. What is PCA and how does it reduce dimensions?

PCA (Principal Component Analysis) is a linear dimensionality reduction technique. It works by finding the directions (eigenvectors of the covariance matrix) along which the data has the highest variance — called principal components. The first principal component is the direction of maximum variance, the second is orthogonal to the first and has the second-highest variance, and so on. By projecting the original 14-feature data onto just the top 2 principal components, we compress the data while retaining as much variance (information) as possible. The 2D projection can then be plotted as a scatter chart to visually validate cluster separation.

Q15. What is the difference between supervised, unsupervised, and semi-supervised learning?

In supervised learning the training data contains both input features and labelled output targets (e.g. predicting churn: yes/no). The model learns a mapping from inputs to outputs. In unsupervised learning there are no labels — the algorithm discovers hidden structure on its own, as KMeans does here by finding natural groupings in user behaviour data. In semi-supervised learning only a small portion of the data is labelled; the model uses the unlabelled data to improve generalisation. This project is fully unsupervised — we never told the algorithm which users are 'High Engagement'; it determined that purely from the feature patterns.

Q16. What is the Elbow Method and how do you interpret the graph?

The Elbow Method plots the KMeans inertia (Y-axis) against the number of clusters K (X-axis). As K increases from 1 to 10, inertia always decreases because more clusters means each cluster contains fewer, more similar points. However, the rate of decrease slows dramatically once the data has been meaningfully separated. The 'elbow' — visually resembling the bend in a human arm — is the K value where the curve transitions from steep to shallow. Choosing K at the elbow gives the best balance between model simplicity (fewer clusters) and cluster quality (low inertia). In this project the elbow at K=4 was used to justify the four-segment user model.

Q17. What is Euclidean distance and why does it matter in KMeans?

Euclidean distance is the straight-line distance between two points in n-dimensional space, computed as the square root of the sum of squared differences across all dimensions: $d = \sqrt{\sum (x_i - y_i)^2}$. In KMeans, every user is assigned to the cluster whose centroid has the smallest Euclidean distance. This means KMeans inherently assumes clusters are spherical and equally sized. It also explains why feature scaling is mandatory — a feature with values 0–500 contributes 50x more to Euclidean distance than a feature with values 0–10, completely distorting cluster assignments without scaling.

Section D: Streamlit, Visualisation & Tools

Q18. How does Streamlit's execution model work? Why does the whole script re-run?

Streamlit uses a top-to-bottom reactive execution model. When the app loads, the script runs once from line 1 to the end. Every time a user interacts with a widget (changes a multiselect, moves a slider), Streamlit re-runs the entire script again from the top. Variables are recomputed with the new widget values. This model is simple but powerful — the developer does not need to write callbacks or event handlers. The `@st.cache_data` decorator is essential for functions like `load_data()` that should not re-execute on every rerun, as it returns the cached result instantly when the inputs haven't changed.

Q19. What is the difference between `plt.savefig()` and `st.pyplot()`? When would you use each?

`plt.savefig('path/file.png')` writes the matplotlib figure to an image file on disk — used in the EDA and clustering scripts (03, 05) to create persistent output files that can be included in reports, presentations, or documentation outside the application. `st.pyplot(fig)` takes a matplotlib figure object held in memory and renders it directly inside the Streamlit webpage as an inline image — no file is written to disk. Use `savefig()` when you need a reusable image file; use `st.pyplot()` when building an interactive dashboard where charts need to update dynamically based on user filter selections.

Q20. What does `plt.figure()` do and why is `plt.close()` necessary?

`plt.figure()` creates a new blank Figure object in matplotlib's state machine, ensuring that subsequent plotting commands draw on a fresh canvas rather than adding to the previously created chart. Without calling `plt.figure()` between charts, matplotlib would attempt to overlay new plots on top of the existing one. `plt.close()` releases the memory occupied by the figure object after it has been saved or displayed. In a script that generates 10+ charts in sequence (like `03_EDA.py`), failing to call `plt.close()` would keep all figures in memory simultaneously, eventually causing `MemoryError` or consuming excessive RAM.

Q21. What is seaborn and how does it relate to matplotlib?

Seaborn is a high-level statistical data visualisation library built on top of matplotlib. It provides a more concise API for common statistical charts (histograms, box plots, heatmaps, pair plots) and automatically handles details like colour palettes, axis scaling, and legend placement. While matplotlib gives full low-level control (drawing individual lines, patches, and text), seaborn lets you produce polished charts with a single function call. In this project, `sns.histplot()`, `sns.scatterplot()`, and `sns.heatmap()` are used for statistical charts, while raw matplotlib `plt.plot()` and `.plot(kind='bar')` are used for simpler bar and pie charts.

Q22. What is the purpose of `random_state=42` in KMeans and sklearn functions?

`random_state` sets the seed for Python's random number generator used internally by the algorithm. KMeans initialises its starting centroids randomly — different starting positions can lead to different final clusters (a local optimum problem). Setting `random_state=42` ensures that every time you run the code on the same data, you get exactly the same cluster assignments. This is critical for reproducibility: other developers, reviewers, or automated tests can run the script and verify they get identical results. The value 42 has no special mathematical significance — it is simply a convention in the data science community.

Q23. What is the `fit()` vs `transform()` vs `fit_transform()` distinction in sklearn?

`fit(X)` trains (learns parameters from) the data — for `StandardScaler` it computes the mean and standard deviation of each column; for KMeans it finds the optimal centroids. `transform(X)` applies the learned parameters to data — for `StandardScaler` it subtracts the mean and divides by std for each value. `fit_transform(X)` is a convenience method that performs both in sequence and is slightly more efficient. The critical rule: always `fit()` on training data only, then `transform()` both training and test data using the same fitted scaler. Fitting on test data would cause data leakage — incorporating information from test samples into the

preprocessing.

Q24. How would you evaluate whether the KMeans clustering is good?

Several evaluation approaches exist. (1) Elbow Method: check that $K=4$ is clearly at the elbow of the inertia curve. (2) Silhouette Score: ranges from -1 to +1; values near +1 indicate well-separated, compact clusters; values near 0 indicate overlapping clusters. (3) PCA Scatter Plot: visually inspect whether the four cluster blobs in the 2D PCA chart are separated with minimal overlap. (4) Cluster Profiling: examine whether the cluster profiles make business sense — High Engagement should have high sessions and low churn risk; Low Engagement should have high churn risk and few sessions. (5) Business Validation: present findings to domain experts to confirm the segments are actionable.

Q25. What is the difference between KMeans, DBSCAN, and Hierarchical Clustering?

KMeans partitions data into exactly K spherical clusters by minimising inertia — fast, scalable, but requires specifying K and assumes convex cluster shapes. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) finds arbitrarily-shaped clusters based on density — it does not require K and automatically labels outliers as noise, making it robust to irregular cluster shapes. Hierarchical Clustering builds a tree (dendrogram) of nested clusters bottom-up (agglomerative) or top-down (divisive) — it does not require K upfront and the dendrogram helps visualise cluster relationships, but it is $O(n^2)$ in memory and cannot scale to large datasets. KMeans was chosen here because user behaviour clusters are expected to be roughly spherical and the dataset is large.

Q26. What is a DataFrame vs a Series in Pandas?

A Series is a one-dimensional labelled array — essentially a single column with an index. `df['engagement_score']` returns a Series. A DataFrame is a two-dimensional labelled data structure — a collection of Series sharing the same index, like a table with rows and columns. `df[['engagement_score', 'churn_risk_score']]` returns a DataFrame with two columns. Most Pandas operations work on both, but some functions like `.corr()` and `.to_csv()` are DataFrame-specific, while `.value_counts()` and `.unique()` are Series-specific. Understanding this distinction is essential for writing correct, bug-free data manipulation code.

Q27. What is feature engineering and could it improve this project?

Feature engineering is the process of creating new informative features from existing ones to improve model performance. In this project, potential engineered features could include: `engagement_per_session = engagement_score / sessions_per_week` (intensity of engagement per visit), `recency_weighted_engagement = engagement_score * (1 / (days_since_last_login + 1))` (penalising users who haven't logged in recently), or `notification_open_rate = notifications_opened_per_week / sessions_per_week`. These derived features might capture user behaviour dimensions not present in the raw data and could produce more meaningful or sharply-separated clusters.

Q28. What is overfitting and does it apply to unsupervised learning like KMeans?

Overfitting in supervised learning is when a model learns the training data too specifically — memorising noise rather than general patterns — and performs poorly on new data. In KMeans, the analogous problem is choosing too large a K : with $K=n$ (one cluster per user), inertia is zero because every user is their own centroid, but the model is completely useless for generalisation or business insights. The Elbow Method guards against this by identifying the K where additional clusters stop providing meaningful structure. Unlike supervised learning, KMeans has no test set — validation comes from the elbow curve, silhouette score, and business sense-checking of the cluster profiles.

Q29. What is the role of the with statement in Python? Is it used in this project?

The with statement is a context manager that automatically handles setup and teardown operations — most commonly ensuring that resources like files, database connections, or locks are properly closed even if an error occurs inside the block. The syntax with open('file') as f: guarantees the file is closed when the block exits, equivalent to a try/finally block. In this project the with col1: syntax in Streamlit uses a context manager to route all plotting commands inside that block to the first column. The with statement does not appear for file I/O here because pd.read_csv() and .to_csv() manage their own file handles internally.

Q30. How would you deploy this Streamlit app to make it accessible online?

The simplest deployment option is Streamlit Community Cloud (streamlit.io/cloud) — free for public repositories. You push the code to GitHub, connect the repository to Streamlit Cloud, set the main file to 07_streamlit_app.py, and the app is live at a public URL within minutes. For production use, the app can be containerised with Docker and deployed to cloud platforms like AWS EC2, Google Cloud Run, or Azure App Service. The file path in load_data() would need to change from a local Windows path to a relative path or cloud storage URL (e.g. AWS S3) so the CSV is accessible in the deployment environment.